



Tecnatura Superior en Desarrollo de Software

Práctica 03 — Excepciones de Dominio y Fail Fast

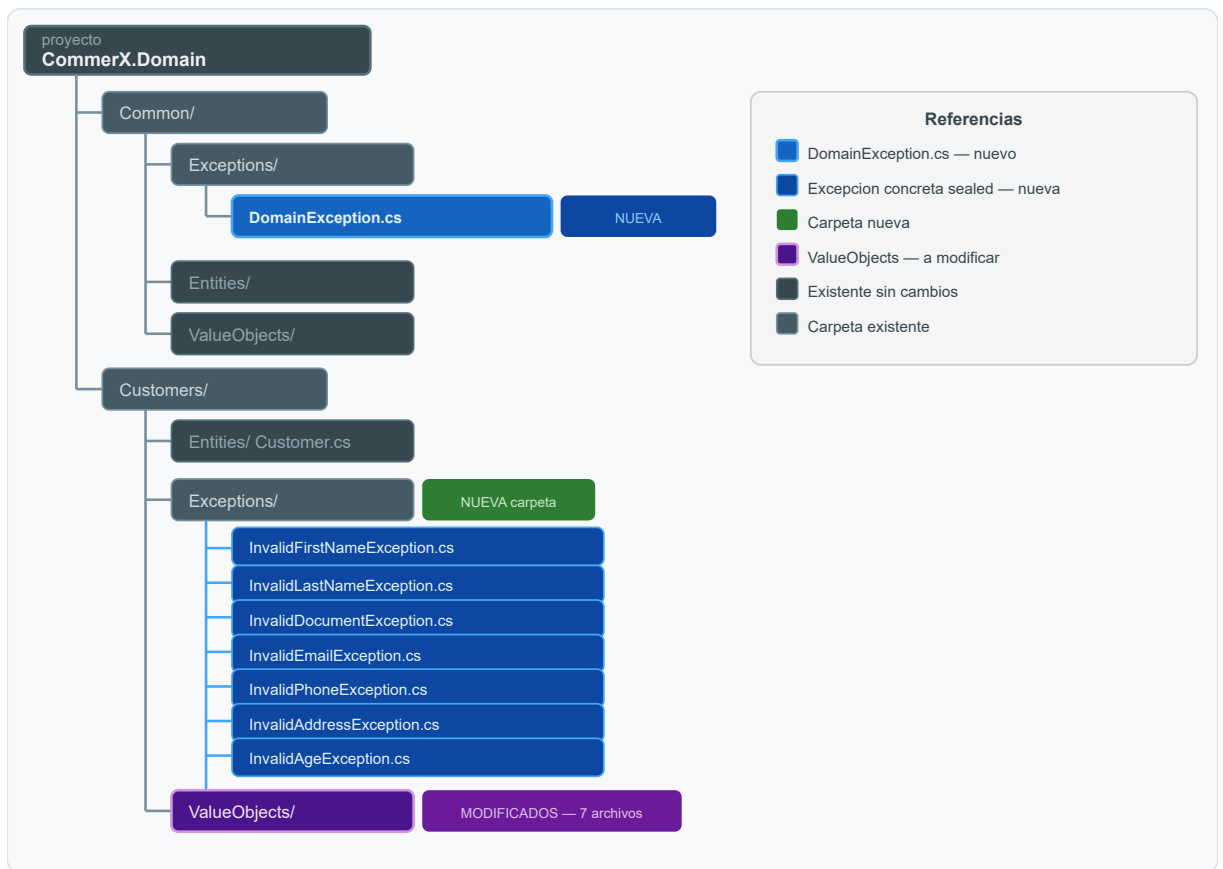
Asignatura: Programación II | Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat | Ciclo Lectivo 2026

Ejemplo práctico

En la Práctica 02 los Value Objects lanzaban `ArgumentException` — una excepción genérica del framework que no pertenece al lenguaje del dominio. En esta práctica se reemplaza ese mecanismo por una jerarquía de excepciones propias del proyecto **CommerX**, aplicando el principio **Fail Fast** con tipos semánticamente correctos.

Objetivo: Crear la clase base `DomainException`, implementar las 7 excepciones personalizadas del módulo Customer y actualizar cada Value Object para lanzar su excepción específica.

Archivos a crear y modificar



Archivo 1 — CommerX.Domain / Common / Exceptions / DomainException.cs

¿Qué hace esta clase?

Es la base abstracta de todas las excepciones del dominio. Al ser `abstract` no puede instanciarse directamente. Su único propósito es servir como tipo común para que Application pueda capturar cualquier error de dominio con un único `catch (DomainException)`.

```

1 namespace CommerX.Domain.Common.Exceptions;
2
3 // abstract: no puede instanciarse directamente – solo sus subclases concretas
4 // hereda de Exception: compatible con el mecanismo estándar de .NET
5 public abstract class DomainException : Exception
6 {
7     // protected: solo accesible desde las subclases – no desde código externo
8     // recibe el mensaje ya construido por la subclase y lo delega a Exception
9     protected DomainException(string message) : base(message) { }
10 }
```

Archivo 2 — CommerX.Domain / Customers / Exceptions /**InvalidFirstNameException.cs****Patrón de toda excepción concreta:**

`sealed class` que hereda de `DomainException`. Constructor público que recibe el valor inválido e incluye ese valor en el mensaje. Sin lógica adicional — solo transporta información sobre el error.

```
1 using CommerX.Domain.Common.Exceptions;
2
3 namespace CommerX.Domain.Customers.Exceptions;
4
5 // sealed: no puede heredarse – su semántica es fija y completa
6 public sealed class InvalidFirstNameException : DomainException
7 {
8     // recibe el valor inválido para incluirlo en el mensaje de diagnóstico
9     public InvalidFirstNameException(string value)
10         : base($"El nombre '{value}' no es valido: minimo 2 caracteres, maximo 50.") { }
11 }
```

Archivo 3 — CommerX.Domain / Customers / ValueObjects / `FirstName.cs` (actualizado)**Cambio aplicado:**

Se reemplaza `throw new ArgumentException(...)` por `throw new InvalidFirstNameException(value)`. La estructura del VO — constructor privado, propiedad inmutable, método `Create()` — permanece exactamente igual.

```
1 using CommerX.Domain.Common.ValueObjects;
2 using CommerX.Domain.Customers.Exceptions;
3
4 namespace CommerX.Domain.Customers.ValueObjects;
5
6 public sealed record FirstName : ValueObject
7 {
8     public string Value { get; }
9
10     private FirstName(string value)
11     {
12         Value = value;
13     }
14
15     public static FirstName Create(string value)
16     {
17         // Fail Fast: nulo o vacío – excepción personalizada de dominio
18         if (string.IsNullOrEmpty(value))
19             throw new InvalidFirstNameException(value ?? "");
20
21         // normalización antes de continuar las validaciones
22         string trimmed = value.Trim();
23
24         // mínimo 2 caracteres sin espacios, máximo 50
25         if (trimmed.Replace(" ", "").Length < 2 || trimmed.Length > 50)
```

```

26         throw new InvalidFirstNameException(trimmed);
27
28         // todas las reglas pasaron – se construye el VO
29         return new FirstName(trimmed);
30     }
31 }

```

Los archivos restantes siguen el mismo patrón.

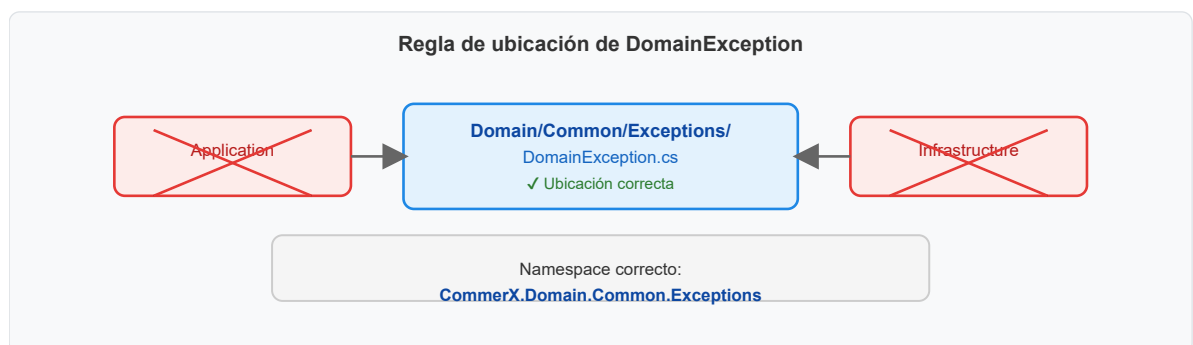
En la actividad implementarás las 6 excepciones restantes y actualizarás los 6 VOs correspondientes. El único que varía en firma es `InvalidAgeException`, que recibe `DateOnly` en lugar de `string`.

Explicación paso a paso

La migración de `ArgumentException` a excepciones de dominio se realiza en tres etapas bien diferenciadas. Cada etapa debe compilar correctamente antes de pasar a la siguiente.

Paso 1 — Crear la clase base `DomainException`

Es el primer archivo que debe existir. Todas las excepciones concretas dependen de él. Si no compila, nada más puede avanzar. Su estructura es mínima: clase `abstract`, hereda de `Exception`, constructor `protected` que recibe el mensaje y lo delega con `: base(message)`.



Paso 2 — Implementar las 7 excepciones concretas

Cada excepción tiene exactamente la misma estructura: `sealed class`, hereda de `DomainException`, un único constructor público que recibe el valor inválido y construye el mensaje. El mensaje debe incluir siempre el valor recibido para facilitar el diagnóstico.

Clase	Parámetro del constructor	Regla que representa
<code>InvalidFirstNameException</code>	<code>string value</code>	Nombre: mínimo 2 chars, máximo 50

Clase	Parámetro del constructor	Regla que representa
<code>InvalidLastNameException</code>	<code>string value</code>	Apellido: mínimo 2 chars, máximo 50
<code>InvalidDocumentException</code>	<code>string value</code>	Documento: solo dígitos, 7-8 chars
<code>InvalidEmailException</code>	<code>string value</code>	Email: formato válido con @
<code>InvalidPhoneException</code>	<code>string value</code>	Teléfono: 6-15 dígitos
<code>InvalidAddressException</code>	<code>string value</code>	Dirección: no vacía, máximo 200 chars
<code>InvalidAgeException</code>	<code>DateOnly birthDate</code>	Edad: mayor de 18 años cumplidos

Diferencia en `InvalidAgeException` :

El parámetro es `DateOnly birthDate` — no `string`. El mensaje se construye formateando la fecha: `$"La fecha '{birthDate:dd/MM/yyyy}' no cumple la regla de mayoría de edad."`

Paso 3 — Actualizar los 7 Value Objects

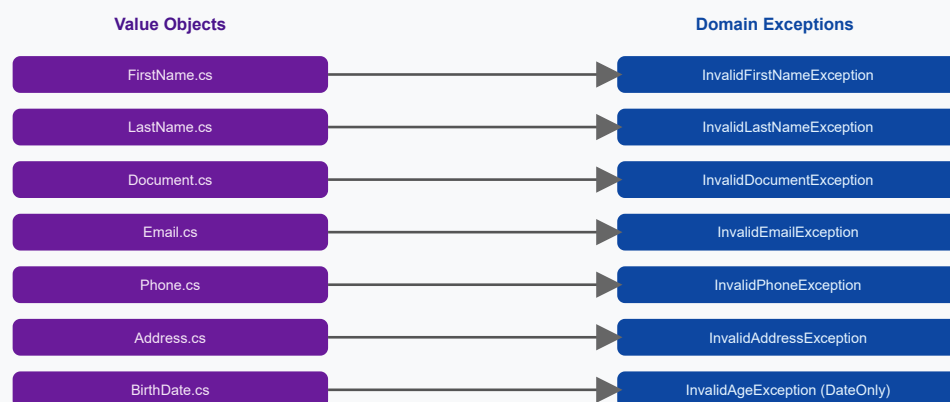
En cada VO se realizan exactamente dos cambios — nada más:

1. Agregar el `using` del namespace de excepciones:

```
using CommerX.Domain.Customers.Exceptions;
```
2. Reemplazar cada `throw new ArgumentException(...)` por el tipo específico correspondiente:

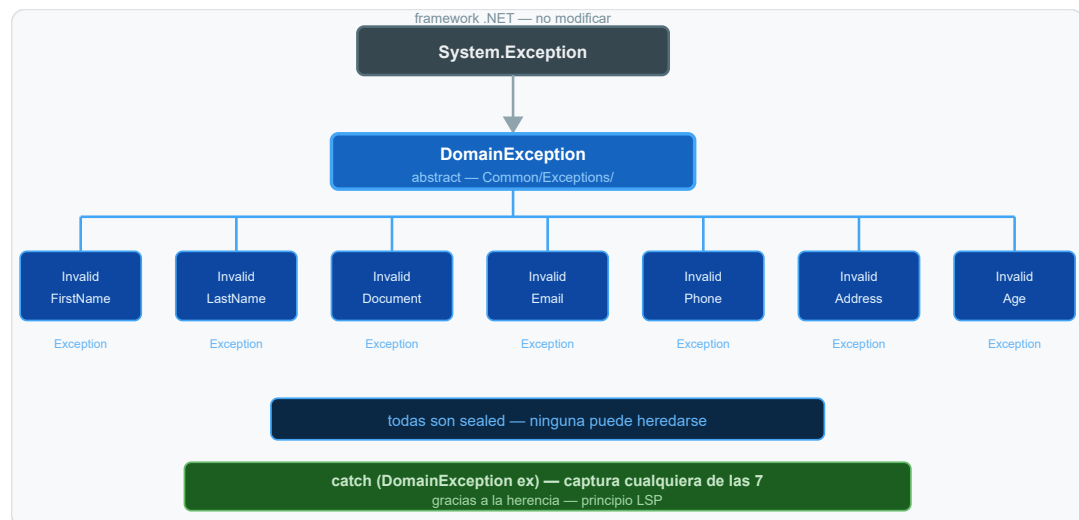
```
throw new InvalidFirstNameException(value);
```

Correspondencia Value Object → Excepción de Dominio



Paso 4 — Verificar la jerarquía completa

Una vez implementadas todas las clases, la jerarquía de herencia debe verse de la siguiente manera. Verificá que cada clase tenga el modificador correcto antes de continuar con la actividad.



Actividad

Implementarás la jerarquía completa de excepciones del caso de uso **CU-CLI-001 — Alta de Cliente** y actualizarás los 7 Value Objects para que utilicen sus excepciones específicas de dominio. Seguí el orden de las partes: cada una depende de la anterior.

Parte A — Clase base DomainException

1. Crear la carpeta `Common/Exceptions/` dentro del proyecto `CommerX.Domain`.
2. Crear el archivo `DomainException.cs`.
3. Declarar el namespace correcto: `CommerX.Domain.Common.Exceptions`.
4. Implementar la clase: `public abstract class DomainException : Exception` con constructor `protected` que recibe `string message` y lo delega con `base(message)`.
5. Compilar el proyecto — debe compilar sin errores antes de continuar.

Parte B — Excepciones de texto (string value)

Crear la carpeta `Customers/Exceptions/` e implementar las siguientes 6 excepciones. Todas reciben `string value` y siguen el mismo patrón que `InvalidFirstNameException` del ejemplo.

B.1 — InvalidFirstNameException.cs

Ya está provista como ejemplo. Copiala y verificá que el namespace sea correcto.

B.2 — InvalidLastNameException.cs

1. Declarar `public sealed class InvalidLastNameException : DomainException`.
2. Constructor que recibe `string value`.
3. Mensaje que incluya el valor y la regla: mínimo 2, máximo 50 caracteres.

B.3 — InvalidDocumentException.cs

1. Declarar `public sealed class InvalidDocumentException : DomainException`.
2. Constructor que recibe `string value`.
3. Mensaje que incluya el valor y la regla: solo dígitos, entre 7 y 8 caracteres.

B.4 — InvalidEmailException.cs

1. Declarar `public sealed class InvalidEmailException : DomainException`.
2. Constructor que recibe `string value`.
3. Mensaje que incluya el valor y la regla: formato email válido.

B.5 — InvalidPhoneException.cs

1. Declarar `public sealed class InvalidPhoneException : DomainException`.
2. Constructor que recibe `string value`.
3. Mensaje que incluya el valor y la regla: entre 6 y 15 dígitos.

B.6 — InvalidAddressException.cs

1. Declarar `public sealed class InvalidAddressException : DomainException`.
2. Constructor que recibe `string value`.
3. Mensaje que incluya el valor y la regla: no vacía, máximo 200 caracteres.

Parte C — Excepción de fecha (`DateOnly birthDate`)**C.1 — InvalidAgeException.cs**

1. Declarar `public sealed class InvalidAgeException : DomainException`.
2. El constructor recibe `DateOnly birthDate` — no `string`.
3. Construir el mensaje formateando la fecha:
`$"La fecha '{birthDate:dd/MM/yyyy}' no cumple la regla de mayoría de edad (18 años)."`

Compilar después de cada parte. No avanzar a la Parte D hasta que todas las excepciones compilen sin errores. Cada excepción mal declarada bloqueará la compilación de los VOs.

Parte D — Actualizar los 7 Value Objects

Para cada uno de los 7 VOs realizá exactamente estos dos cambios. No modificar la estructura del VO — solo los `throw`.

1. Agregar en la parte superior del archivo:

```
using CommerX.Domain.Customers.Exceptions;
```

2. Reemplazar cada `throw new ArgumentException(...)` por el tipo específico usando el valor como argumento.

Archivo VO	Reemplazar por	Argumento
FirstName.cs	InvalidFirstNameException	value ?? "" / trimmed
LastName.cs	InvalidLastNameException	value ?? "" / trimmed
Document.cs	InvalidDocumentException	value ?? "" / trimmed
Email.cs	InvalidEmailException	value ?? "" / normalizado
Phone.cs	InvalidPhoneException	value ?? "" / trimmed
Address.cs	InvalidAddressException	value ?? "" / trimmed
BirthDate.cs	InvalidAgeException	value (DateOnly — sin ?? "")

Parte E — Verificación en Program.cs

Escribí el siguiente bloque en `Program.cs` para verificar que el circuito completo funciona correctamente.

1. Probar con email inválido y capturar el tipo específico:

```
try { Email.Create("noesvalido"); }
catch (InvalidEmailException ex) { Console.WriteLine(ex.Message); }
```

Verificar que el mensaje incluye el valor "noesvalido".

2. Probar con fecha de nacimiento de menor de edad:

```
BirthDate.Create(DateOnly.FromDateTime(DateTime.Today.AddYears(-10)))
```

Capturar `InvalidAgeException` y verificar que el mensaje incluye la fecha.

3. Probar con documento inválido y capturar usando la base:

```
catch (DomainException ex) { Console.WriteLine(ex.Message); }
```

Verificar que `DomainException` captura `InvalidDocumentException`.

Resultado esperado:

Cada mensaje de error identifica con precisión el valor inválido y la regla violada. El tipo `DomainException` captura cualquiera de las 7 excepciones. El proyecto compila sin errores ni warnings.

Parte F — Preguntas de verificación

1. ¿Por qué `DomainException` es `abstract`? ¿Qué ocurriría si no lo fuera?
2. ¿Por qué cada excepción concreta es `sealed`?
3. ¿Por qué el constructor de `DomainException` es `protected` y no `public`?
4. ¿Qué ventaja tiene `InvalidDocumentException` sobre `ArgumentException` cuando el sistema crece a 50 VOs distintos?
5. Al capturar `DomainException` en vez de `InvalidEmailException`, ¿perdés información útil? ¿En qué capa es aceptable usar la captura genérica?
6. ¿En qué capa del proyecto viven las excepciones? ¿Por qué no pueden estar en `Application`?

Parte G — Reflexión final

Un compañero propone: *"Para no duplicar código, voy a poner todas las excepciones en una sola clase `DomainException` con un enum que indique el tipo de error."*

- ¿Qué principio SOLID viola esa propuesta?
- ¿Cómo impacta en la capacidad de capturar errores específicos en `Application`?
- ¿Qué desventaja tendría al agregar un nuevo VO al sistema?
- Escribí un argumento técnico de dos párrafos para defender la jerarquía actual.

Criterios de evaluación

Compilación: toda la solución compila sin errores ni warnings.

DomainException: es `abstract`, hereda de `Exception`, constructor `protected`.

Excepciones concretas: son `sealed`, heredan de `DomainException`, mensaje incluye el valor.

InvalidAgeException: recibe `DateOnly` y formatea la fecha en el mensaje.

VOs actualizados: usan la excepción específica — sin `ArgumentException`.

Verificación: los tres casos de prueba en `Program.cs` funcionan correctamente.

Reflexión: las respuestas demuestran comprensión del principio Fail Fast y de la jerarquía DDD.



Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat | Tecnicatura Superior en Desarrollo de Software | Asignatura:
Programación II | Tema: Práctica 03 — Excepciones de Dominio y Fail Fast | Versión: 2026